

# MI VDEC API

---

## 1 Overview

---

### 1.1. Module Introduction

Video decoding function reference, providing functions such as decoding channel creation, stream transmission and control, output image scaling and cropping.

There are two main types of input sources for the VDEC module:

- The user reads the stream file and sends data to the decoding module;
- The user directly sends the stream data received by the network to the decoding module.

Table 1-1 Chip decoding specifications

| chip     | hardware decoding module | Maximum number of channels supported | supporting agreement | Resolution range                                                 | Maximum performance                              |
|----------|--------------------------|--------------------------------------|----------------------|------------------------------------------------------------------|--------------------------------------------------|
| Taiyaki  | DEC2.0                   | 16                                   | H.264/H.265          | H.264: max 8192x8192, min 32x32<br>H.265: max 8192x8192, min 8x8 | H.264: 3840x2160@60FPS<br>H.265: 3840x2160@30FPS |
| Takoyaki | DEC2.0                   | 4                                    | H.264/H.265          | H.264: max 8192x8192, min 32x32<br>H.265: max 8192x8192, min 8x8 | H.264: 920x1080@60FPS<br>H.265: 1920x1080@30FPS  |

## 1.2. Decoding Flowchart

### 1.2.1. Taiyaki data flow diagram

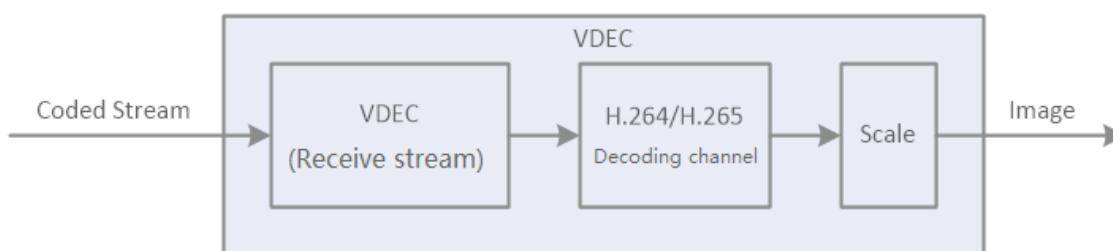


Figure 1-1 VDEC data flow chart

### 1.2.2. Takoyaki data flow diagram

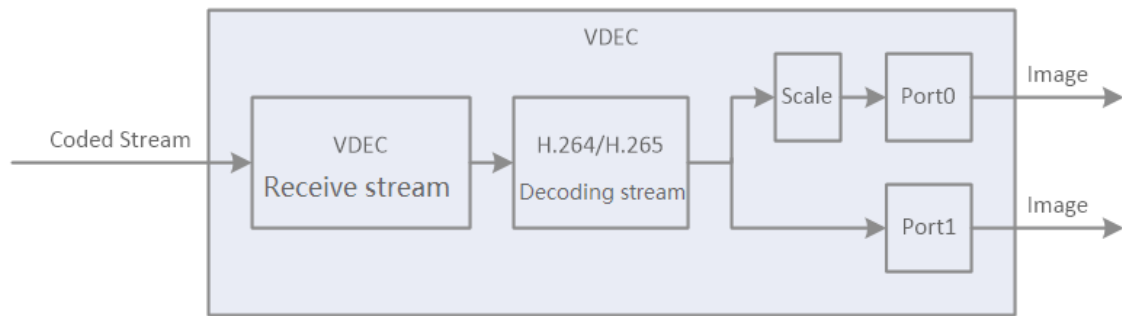


Figure 1-2 VDEC data flow chart

### 1.3. Keyword Description

- Stream sending method

The VDEC decoder provides two stream transmission methods:

- Send by stream (E\_MI\_VDEC\_VIDEO\_MODE\_STREAM): The user can send a stream of any length to the decoder each time, and the parsing and splicing process of the stream frame is completed by the decoder. For H.264/H.265, it is necessary to receive the next frame of code stream to confirm the end of the current code stream frame, so in this mode, inputting a frame of H.264/H.265 code stream will not immediately Solve the image.
- Send by frame (E\_MI\_VDEC\_VIDEO\_MODE\_FRAME): The user sends a complete frame of code stream to the decoder each time, and each time the sending interface is called, the decoder considers the frame code stream has ended and starts to decode the image. Therefore, the user needs to ensure that the code stream sent each time must be a complete frame, otherwise decoding errors will occur. Using this mode can achieve the purpose of fast decoding.
- The code stream transmission mode eVideoMode can be set in the interface [MI\\_VDEC\\_CreateChn](#) [#24-mi\_vdec\_createchn] .
- Image output order
  - According to the H.264/H265 protocol, the decoded image may not be output immediately after decoding. The VDEC decoder can achieve the

purpose of outputting as soon as possible by setting different image output methods. Image output methods include the following two:

- decoding sequence
  - The decoded images are output in the order of decoding.
- display order
  - The decoded images are output in the order in which they are displayed in the protocol.
  - 根据H.264/H.265协议，视频的解码顺序（即解码序）和解码图像的显示顺序（即显示序）可能不一致。例如：解码B帧时，需要前后的P帧作为参考，所以B帧后的P帧先于B帧解码，但B帧先于P帧输出。按解码序输出是保证快速输出的一个必要条件，用户选择按解码序输出，需保证码流的解码序与显示序相同。
  - 按帧发送码流与按解码序输出相结合能达到快速解码和快速输出的目的，用户必须保证每次发送的是完整的一帧码流以及码流的解码序和显示序相同。
  - 图像输出方式eOutputOrder可在接口MI\_VDEC\_SetChnParam [#212-mi\_vdec\_setchnparam] 中设置。
- 时间戳（PTS）处理
  - 在按帧模式发送码流时，解码输出的图像时间戳PTS为发送码流接口(MI\_VDEC\_SendStream [#214-mi\_vdec\_sendstream])中用户送入的PTS，解码器不会更改此值；如果用户配置的PTS值为0，则表示用户不进行帧率控制，而是由视频显示模块（DISP）进行帧率控制；如果用户送入的PTS值为-1，则表示此图像不会被视频显示模块（DISP）显示；如果是其他值，则表示视频显示模块（DISP）根据用户设置的PTS值进行帧率控制。

**注意：不能出现PTS值为0和非0混合的情况。**

- 在按流模式下发送码流时，解码输出图像的PTS统一设为0，表示用户不进行帧率控制，而是由视频显示模块（DISP）进行帧率控制。
- 码流Buffer
  - 码流Buffer用于缓存用户输入的码流，然后送入解码器解码。码流buffer的大小没有固定的计算公式，一般按照经验值，分辨率小于或等于D1,建议配置为512KB；分辨率在(D1,1080P],建议配置为1MB；分辨率在

(1080P, 4K], 建议配置为2MB; 用户可根据实际需求, 选择合适的值设置。

- **eDpbBufMode**

- **eDpbBufMode**可供用户设置不同的buffer模式来进行解码。根据不同的场景, 来选择不同的buffer模式, 可以达到省内存的目的。当设为 **E\_MI\_VDEC\_DPB\_MODE\_NORMAL**, 不能节省内存; 当设为 **E\_MI\_VDEC\_DPB\_MODE\_INPLACE\_ONE\_BUF**, 用于解码仅包含一个参考帧的码流类型, 此时仅需一个DPB buffer即可; 当设为 **E\_MI\_VDEC\_DPB\_MODE\_INPLACE\_TWO\_BUF**, 用于解码仅包含两个参考帧的码流类型, 此时仅需两个DPB buffer即可。

- **u32RefFrameNum**

- **u32RefFrameNum**表示最大参考帧个数。由于在实际的场景中, 系统的内存并不是无限大, 用户可以根据产品的定义对参考帧的个数进行限制, 以便提示用户当前码流可能超规格。如果用户设置了**eDpbBufMode**为 **E\_MI\_VDEC\_DPB\_MODE\_INPLACE\_ONE\_BUF**或者 **E\_MI\_VDEC\_DPB\_MODE\_INPLACE\_TWO\_BUF**, 则该参数无效。如果用户设置了**eDpbBufMode**为 **E\_MI\_VDEC\_DPB\_MODE\_NORMAL**, 则解码器在参考帧个数受到限制时进行解码, 解码图像可能会异常或者解码超时。

- **输出图像缩放**

- 用户调用**MI\_VDEC\_SetOutputPortAttr** [#219-mi\_vdec\_setoutputportattr] 可以对输出的图像进行缩放, 以输出需要的分辨率的图像。但需注意, 解码器仅支持对图像缩小, 不支持对图像放大。

- **低延时**

- 这里的延时是指解码B帧后, 需要对待显示的图像队列进行重排序 ( **reorder** ) 所产生的延时。当解码不含B帧的码流时, 解码顺序和显示顺序相同, 因此不会出现延时输出, 即低延时; 而解码含B帧的码流时, 由于B帧有前后向参考, 需要等待所参考的图像解码完后, 才能解当前帧, 并且需要对解码后的图像重排序 ( **reorder** ), 由此产生输出延时。这两种模式需要通过设置**bDisableLowLatency**来完成。默认 **bDisableLowLatency**为**FALSE**, 用户无需设置; 当需要解码B帧码流时, 需要将**bDisableLowLatency**设为**TRUE**。需要注意, **bDisableLowLatency**为**TRUE**时, 不能用于解码非B帧码流。

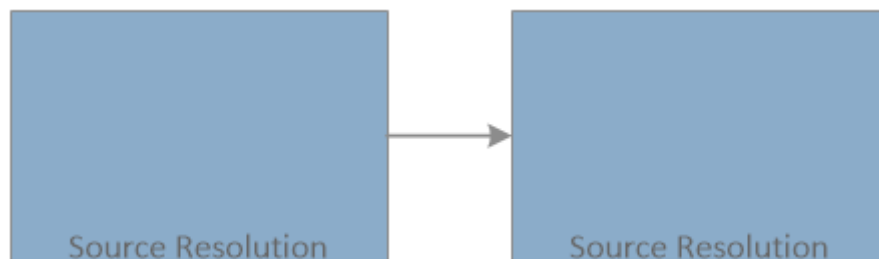
- **输出buffer模式**

- 用户可以调用MI\_VDEC\_SetOutputPortLayoutMode [#221-mi\_vdec\_setoutputportlayoutmode]设置输出buffer的模式。该接口仅使用于芯片Takoyaki。当用户需要手动控制输出buffer的模式时，可调用该接口。当为E\_MI\_VDEC\_OUTBUF\_LAYOUT\_LINEAR模式时，表示输出buffer为线性模式；当为E\_MI\_VDEC\_OUTBUF\_LAYOUT\_TILE模式时，表示输出buffer为TILE模式；默认为E\_MI\_VDEC\_OUTBUF\_LAYOUT\_AUTO模式，表示输出buffer会自动在线性模式和TILE模式间切换。
- 输出图像裁剪

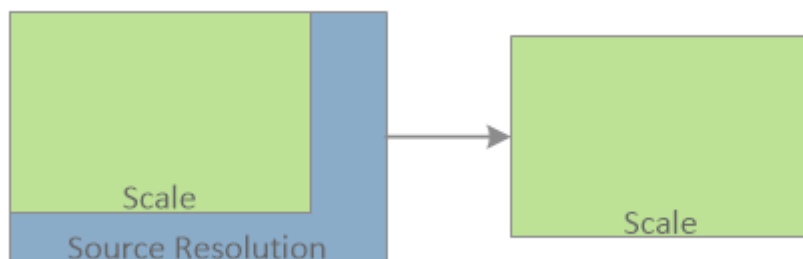
用户调用MI\_VDEC\_SetDestCrop可以对解码后的图像进行裁剪再输出。设置裁剪时，坐标X和宽必须按16对齐，坐标Y和高必须按2对齐，否则接口会返回错误码MI\_ERR\_VDEC\_ILLEGAL\_PARAM。如果裁剪功能和缩放功能同时使用，则裁剪的图像大小必须小于缩放的大小，否则接口会返回错误码MI\_ERR\_VDEC\_ILLEGAL\_PARAM。

裁剪和缩放的关系：

- 缩放关闭，裁剪关闭，输出为原图像；



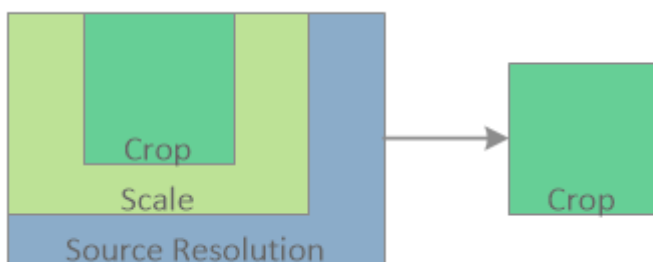
- 缩放开启，裁剪关闭，输出为缩放后的图像；



- 缩放关闭，裁剪开启，输出为裁剪后的图像；



- d. 缩放开启，裁剪开启，输出为以缩放的宽和高为参考，再做裁剪后的图像；如果开启了裁剪，则最后输出的**frame buffer**以裁剪大小为最终输出大小。



## 2. API参考

### 2.1. 功能模块API

表2-1

| API名                                                                | 功能             |
|---------------------------------------------------------------------|----------------|
| <a href="#">MI_VDEC_InitDev</a> [#22-mi_vdec_initdev]               | 设备初始化          |
| <a href="#">MI_VDEC_DeInitDev</a> [#23-mi_vdec_deinitdev]           | 设备去初始化         |
| <a href="#">MI_VDEC_CreateChn</a> [#24-mi_vdec_createchn]           | 创建解码通道         |
| <a href="#">MI_VDEC_DestroyChn</a> [#25-mi_vdec_destroychn]         | 销毁解码通道         |
| <a href="#">MI_VDEC_GetChnAttr</a> [#26-mi_vdec_getchnattr]<br>API名 | 获取解码通道属性<br>功能 |

|                                                                                        |                |
|----------------------------------------------------------------------------------------|----------------|
| <a href="#">MI_VDEC_StartChn</a> [#27-mi_vdec_startchn]                                | 解码通道开始接收码流     |
| <a href="#">MI_VDEC_StopChn</a> [#28-mi_vdec_stopchn]                                  | 解码通道停止接收码流     |
| <a href="#">MI_VDEC_GetChnStat</a> [#29-mi_vdec_getchnstat]                            | 获取解码通道状态       |
| <a href="#">MI_VDEC_FlushChn</a> [#210-mi_vdec_flushchn]                               | 清理解码通道缓存数据     |
| <a href="#">MI_VDEC_ResetChn</a> [#211-mi_vdec_flushchn]                               | 重置解码通道         |
| <a href="#">MI_VDEC_SetChnParam</a> [#212-mi_vdec_setchnparam]                         | 设置解码通道参数       |
| <a href="#">MI_VDEC_GetChnParam</a> [#213-mi_vdec_getchnparam]                         | 获取解码通道参数       |
| <a href="#">MI_VDEC_SendStream</a> [#214-mi_vdec_sendstream]                           | 向解码通道发送码流数据    |
| <a href="#">MI_VDEC_GetUserData</a> [#215-mi_vdec_getuserdata]                         | 获取解码通道的用户数据    |
| <a href="#">MI_VDEC_ReleaseUserData</a> [#216-mi_vdec_releaseuserdata]                 | 释放解码通道的用户数据    |
| <a href="#">MI_VDEC_SetDisplayMode</a> [#217-mi_vdec_setdisplaymode]                   | 设置解码通道显示模式     |
| <a href="#">MI_VDEC_GetDisplayMode</a> [#218-mi_vdec_getdisplaymode]                   | 获取解码通道显示模式     |
| <a href="#">MI_VDEC_SetOutputPortAttr</a> [#219-mi_vdec_setoutputportattr]             | 设置解码通道输出端口属性   |
| <a href="#">MI_VDEC_GetOutputPortAttr</a> [#220-mi_vdec_getoutputportattr]             | 获取解码通道输出端口属性   |
| <a href="#">MI_VDEC_SetOutputPortLayoutMode</a> [#221-mi_vdec_setoutputportlayoutmode] | 设置输出端口Buffer模式 |
| <a href="#">MI_VDEC_GetOutputPortLayoutMode</a> [#222-mi_vdec_getoutputportlayoutmode] | 获取输出端口Buffer模式 |

|                                                |            |
|------------------------------------------------|------------|
| mi_vdec_getoutputportlayoutmodej               | 此          |
| MI_VDEC_SetDestCrop [#223-mi_vdec_setdestcrop] | 设置解码通道裁剪属性 |
| MI_VDEC_GetDestCrop [#224-mi_vdec_getdestcrop] | 获取解码通道裁剪属性 |

2.2. MI\_VDEC\_InitDev

- 功能  
解码设备初始化。
- 定义

```
MI_S32 MI_VDEC_InitDev(MI_VDEC_InitParam_t *pstVdecInitParam);
```

- 形参

表2-2

| 参数名称             | 描述                                                                  | 输入/输出 |
|------------------|---------------------------------------------------------------------|-------|
| pstVdecInitParam | 解码设备初始化参数指针，数据类型:<br>MI_VDEC_InitParam_t [#312-mi_vdec_initparam_t] | 输入    |

- 返回值

表2-3

| 返回值                  | 描述       |
|----------------------|----------|
| MI_SUCCESS           | 成功       |
| MI_ERR_VDEC_FAILED   | 失败       |
| MI_ERR_VDEC_INITED   | 设备已经初始化  |
| MI_ERR_VDEC_NULL_PTR | 输入参数为空指针 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 该接口为可选调用，如果仅使用默认的初始化参数，则无需调用；
  - 若选择使用该接口，则必须在创建解码通道前调用，否则会返回设备已初始化的错误码MI\_ERR\_VDEC\_INITED；
  - 若重复调用该接口，则会返回设备已初始化的错误码MI\_ERR\_VDEC\_INITED。
- 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;

    MI_VDEC_CHN VdecChn = 0;

    MI_VDEC_InitParam_t stInitParam;

    MI_VDEC_ChnAttr_t stChnAttr;

    memset(&stInitParam, 0x0, sizeof(MI_VDEC_InitParam_t));

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

    stInitParam.bDisableLowLatency = FALSE;
```

```
s32Ret = MI_VDEC_InitDev(&stInitParam);

if(MI_SUCCESS != s32Ret)

{
    printf("MI_VDEC_InitDev failed, s32Ret: 0x%x.\n",
s32Ret);

    return s32Ret;

}

VdecChn = 0;

stChnAttr.eCodecType      = E_MI_VDEC_CODEC_TYPE_H264;

stChnAttr.u32PicWidth     = 1920;

stChnAttr.u32PicHeight    = 1080;

stChnAttr.eVideoMode      = E_MI_VDEC_VIDEO_MODE_FRAME;

stChnAttr.u32BufSize      = 1024*1024;

stChnAttr.eDpbBufMode     =
E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;

stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;

stChnAttr.u32Priority = 0;

s32Ret = MI_VDEC_CreateChn(VdecChn, &stChnAttr);
if(MI_SUCCESS != s32Ret)

{
    printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n",
s32Ret);

    return s32Ret;

}

return MI_SUCCESS;
}
```

---

## 2.3. MI\_VDEC\_DeInitDev

- 功能

解码设备去初始化。

- 定义

```
MI_S32 MI_VDEC_DeInitDev(void);
```

- 返回值

表2-4

| 返回值                  | 描述     |
|----------------------|--------|
| MI_SUCCESS           | 成功     |
| MI_ERR_VDEC_FAILED   | 失败     |
| MI_ERR_VDEC_NOT_INIT | 设备未初始化 |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 该接口为可选调用；
- 若选择使用该接口，则必须在销毁解码通道后调用，否则会返回设备未初始化的错误码MI\_ERR\_VDEC\_NOT\_INIT；
- 若重复调用该接口，则会返回设备未初始化的错误码MI\_ERR\_VDEC\_NOT\_INIT。

- 举例

```
MI_S32 StopVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
```

```

MI_VDEC_CHN VdecChn = 0;

s32Ret = MI_VDEC_DestroyChn(VdecChn);

if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n",
s32Ret);

    return s32Ret;
}

//Confirm that you have destroyed all channels
s32Ret = MI_VDEC_DeInitDev();

if(MI_SUCCESS != s32Ret)
{
    printf("MI_VDEC_DeInitDev failed, s32Ret: 0x%x.\n",
s32Ret);

    return s32Ret;
}

return MI_SUCCESS;
}

```

---

## 2.4. MI\_VDEC\_CreateChn

- 功能

创建视频解码通道。

- 定义

```

MI_S32 MI_VDEC_CreateChn(MI_VDEC_CHN VdecChn,MI_VDEC_ChnAttr_t
*pstChnAttr)

```

- 形参

表2-5

| 参数名称       | 描述                                                           | 输入/输出 |
|------------|--------------------------------------------------------------|-------|
| VdecChn    | 解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                       | 输入    |
| pstChnAttr | 解码通道属性指针，数据类型: MI_VDEC_ChnAttr_t<br>[#313-mi_vdec_chnattr_t] | 输入    |

- 返回值

表2-6

| 返回值                       | 描述                                         |
|---------------------------|--------------------------------------------|
| MI_SUCCESS                | 成功                                         |
| MI_ERR_VDEC_FAILED        | 失败                                         |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围，取值范围：[0, MI_VDEC_MAX_CHN_NUM) |
| MI_ERR_VDEC_NOT_INIT      | 模块没有成功加载                                   |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数或输入参数超过通道解码能力                          |
| MI_ERR_VDEC_CHN_EXIST     | 试图创建已经存在的通道                                |
| MI_ERR_VDEC_NOMEM         | 分配内存失败（如系统内存不足）                            |
| MI_ERR_VDEC_NULL_PTR      | 输入参数为空指针                                   |
| MI_ERR_VDEC_NOT_SUPPORT   | 该操作或者功能不支持                                 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h

- 库文件：libmi.a
- 注意
  - 通道号不能超出最大的通道号范围。
- 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_CHN VdecChn = 0;
    MI_VDEC_DisplayMode_e eDisplayMode =
E_MI_VDEC_DISPLAY_MODE_MAX;
    MI_VDEC_ChnAttr_t stChnAttr;
    MI_VDEC_OutputPortAttr_t stOutputPortAttr;

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));
    memset(&stOutputPortAttr, 0x0,
sizeof(MI_VDEC_OutputPortAttr_t));

    VdecChn = 0;
    stChnAttr.eCodecType      = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth    = 1920;
    stChnAttr.u32PicHeight   = 1080;
    stChnAttr.eVideoMode     = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize     = 1024*1024;
    stChnAttr.eDpbBufMode    =
E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
    stChnAttr.u32Priority = 0;

    s32Ret = MI_VDEC_CreateChn(VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;
    }

    eDisplayMode = E_MI_VDEC_DISPLAY_MODE_PLAYBACK;
    s32Ret = MI_VDEC_SetDisplayMode (VdecChn, eDisplayMode);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SetDisplayMode failed, s32Ret:
0x%x.\n", s32Ret);
        return s32Ret;
    }
}
```

```

    s32Ret = MI_VDEC_StartChn(VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_StartChn failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;
    }

    stOutputPortAttr.u16Width  = 720;
    stOutputPortAttr.u16Height = 576;
    s32Ret = MI_VDEC_SetOutputPortAttr(VdecChn,
&stOutputPortAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_SetOutputPortAttr failed, s32Ret:
0x%x.\n", s32Ret);
        return s32Ret;
    }

    stCropCfg.bEnable    = TRUE;

    stCropCfg.stRect.u16X  = 0;

    stCropCfg.stRect.u16Y  = 0;

    stCropCfg.stRect.u16Width = 640;

    stCropCfg.stRect.u16Height = 480;

    s32Ret = MI_S32 MI_VDEC_SetDestCrop(VdecChn, &stCropCfg);

    if(MI_SUCCESS != s32Ret)

    {

        printf("MI_VDEC_SetDestCrop failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;

    }

    return MI_SUCCESS;
}

```

---

## 2.5. MI\_VDEC\_DestroyChn

- 功能  
销毁视频解码通道。
- 定义

```
MI_S32 MI_VDEC_DestroyChn(MI_VDEC_CHN VdecChn);
```

- 形参

表2-7

| 参数名称    | 描述                                       | 输入/输出 |
|---------|------------------------------------------|-------|
| VdecChn | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |

- 返回值

表2-8

| 返回值                       | 描述             |
|---------------------------|----------------|
| MI_SUCCESS                | 成功             |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围   |
| MI_ERR_VDEC_CHN_NOT_STOP  | 停止接收码流前不允许关闭通道 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道不存在          |
| MI_ERR_VDEC_NOT_INIT      | 模块没有成功加载       |
| MI_ERR_VDEC_NOT_SUPPORT   | 该操作或者功能不支持     |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意

- 销毁前需停止接收码流。
- 举例

```
MI_S32 StopVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_CHN VdecChn = 0;

    // Destroy send stream thread
    ...

    s32Ret = MI_VDEC_StopChn(VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_StopChn failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;
    }

    s32Ret = MI_VDEC_DestroyChn(VdecChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_DestroyChn failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;
    }

    return MI_SUCCESS;
}
```

---

## 2.6. MI\_VDEC\_GetChnAttr

- 功能  
获取视频解码通道属性。
- 定义

```
MI_S32 MI_VDEC_GetChnAttr(MI_VDEC_CHN VdecChn, MI_VDEC_ChnAttr_t
*pstChnAttr);
```

- 形参

表2-9

| 参数名称       | 描述                                                              | 输入/输出 |
|------------|-----------------------------------------------------------------|-------|
| VdecChn    | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                        | 输入    |
| pstChnAttr | 解码通道属性指针参数类型: MI_VDEC_InitParam_t<br>[#312-mi_vdec_initparam_t] | 输出    |

- 返回值

表2-10

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道不存在        |
| MI_ERR_VDEC_NOT_INIT      | 模块没有成功加载     |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

2.7. MI\_VDEC\_StartChn

- 功能  
开启解码器。
- 定义

```
MI_S32 MI_VDEC_StartChn(MI_VDEC_CHN VdecChn);
```

- 形参

表2-11

| 参数名称    | 描述                                       | 输入/输出 |
|---------|------------------------------------------|-------|
| VdecChn | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |

- 返回值

表2-12

| 返回值                       | 描述                       |
|---------------------------|--------------------------|
| MI_SUCCESS                | 成功                       |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围             |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或者已销毁               |
| MI_ERR_VDEC_NOT_INIT      | 模块没有加载                   |
| MI_ERR_VDEC_NOT_PERM      | 非法操作，如在进行此操作前必须先禁止插入用户图片 |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 启动接收码流前必须保证通道已创建，否则会返回通道未创建的错误码 MI\_ERR\_VDEC\_CHN\_UNEXIST。
- 启动接收码流前必须保证已经禁止使能用户图片，否则返回该操作不允许的错误码 MI\_ERR\_VDEC\_NOT\_PERM。

- 启动接收码流之后，才能调用 [MI\\_VDEC\\_SendStream](#) [#214-mi\_vdec\_sendstream] 发送码流成功。
- 重复调用启动接收码流接口时，返回成功。
- 举例

请参见[MI\\_VDEC\\_CreateChn](#) [#24-mi\_vdec\_createchn] 的举例。

2.8. MI\_VDEC\_StopChn

- 功能  
停止解码器。
- 语法

```
MI_S32 MI_VDEC_Stop(MI_VDEC_CHN VdecChn);
```

- 形参

表2-13

| 参数名称    | 描述                                       | 输入/输出 |
|---------|------------------------------------------|-------|
| VdecChn | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |

- 返回值

表2-14

| 返回值                       | 描述                   |
|---------------------------|----------------------|
| MI_SUCCESS                | 成功。                  |
| MI_ERR_VDEC_FAILED        | 失败。                  |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。        |
| MI_ERR_VDEC_CHN_NOT_START | 通道未使能                |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或者已销毁。          |
| MI_ERR_VDEC_NOT_INIT      | 系统没有初始化或者相关依赖的模块没有加载 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 调用此接口后，调用发送码流接口 [MI\\_VDEC\\_SendStream](#) [#214-mi\_vdec\_sendstream] 会返回失败。
  - 重复调用停止接收码流接口时，返回MI\_ERR\_VDEC\_CHN\_NOT\_START
- 举例

请参见[MI\\_VDEC\\_DestroyChn](#) [#25-mi\_vdec\_destroychn]的举例。

---

## 2.9. MI\_VDEC\_GetChnStat

- 功能

查询解码通道状态。
- 语法

```
MI_S32 MI_VDEC_GetChnStat (MI_VDEC_CHN
VdecChn,MI_VDEC_ChnStat_t *pstChnStat);
```

- 形参

表2-15

| 参数名称       | 描述                                     | 输入/输出 |
|------------|----------------------------------------|-------|
| VdecChn    | 解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |
| pstChnStat | 解码通道状态结构体指针。                           | 输出    |

- 返回值

表2-16

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_FAILED        | 失败           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 参数指针为空       |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

- 举例

请参见[MI\\_VDEC\\_SendStream](#) [#214-mi\_vdec\_sendstream] 的举例。

---

## 2.10. MI\_VDEC\_FlushChn

- 功能

清理解码通道缓存数据。

- 定义

```
MI_S32 MI_VDEC_FlushChn(MI_VDEC_CHN VdecChn);
```

- 形参

表2-17

| 参数名称    | 描述                                       | 输入/输出 |
|---------|------------------------------------------|-------|
| VdecChn | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |

- 返回值

表2-18

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_FAILED        | 失败           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_CHN_NOT_START | 通道未使能        |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 该接口可用于解码过程中，切换到其他GOP时，清理解码通道缓存的数据，从而使解码继续进行。

## 2.11. MI\_VDEC\_ResetChn

- 功能  
复位解码通道。
- 定义

```
MI_S32 MI_VDEC_ResetChn(MI_VDEC_CHN VdecChn);
```

- 形参

表2-19

| 参数名称    | 描述                                       | 输入/输出 |
|---------|------------------------------------------|-------|
| VdecChn | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |

- 返回值

表2-20

| 返回值                       | 描述                                                                             |
|---------------------------|--------------------------------------------------------------------------------|
| MI_SUCCESS                | 成功                                                                             |
| MI_ERR_VDEC_FAILED        | 失败                                                                             |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围                                                                   |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁                                                                      |
| MI_ERR_VDEC_NOT_PERM      | 无法重置通道，操作不能与 <a href="#">MI_VDEC_SendStream</a> [#214-mi_vdec_sendstream] 同时调用 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

- 注意
  - 该接口暂不支持。

## 2.12. MI\_VDEC\_SetChnParam

- 功能  
设置解码通道参数。
- 语法

```
MI_S32 MI_VDEC_SetChnParam(MI_VDEC_CHN
VdecChn,MI_VDEC_ChnParam_t * pstChnParam);
```

- 形参

表2-21

| 参数名称        | 描述                                       | 输入/输出 |
|-------------|------------------------------------------|-------|
| VdecChn     | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |
| pstChnParam | 通道参数。                                    | 输入    |

- 返回值

表2-22

| 返回值                       | 描述                                                     |
|---------------------------|--------------------------------------------------------|
| MI_SUCCESS                | 成功                                                     |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围                                           |
| MI_ERR_VDEC_NOT_PERM      | 非法操作，操作前需调用MI_VDEC_StopChn [#28-mi_vdec_stopchn]停止通道解码 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 该接口暂不支持。
  - JPEG解码时，不支持调用该接口。

2.13. MI\_VDEC\_GetChnParam

- 功能  
获取解码通道参数。
- 语法

```
MI_S32 MI_VDEC_GetChnParam(MI_VDEC_CHN
VdecChn,MI_VDEC_ChnParam_t * pstChnParam);
```

- 形参

表2-23

| 参数名称        | 描述                                       | 输入/输出 |
|-------------|------------------------------------------|-------|
| VdecChn     | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM) | 输入    |
| pstChnParam | 通道参数                                     | 输出    |

- 返回值

表2-24

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 参数不合法        |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 该接口暂不支持。

---

## 2.14. MI\_VDEC\_SendStream

- 功能

向解码通道发送码流数据。
- 语法

```
MI_S32 MI_VDEC_SendStream(MI_VDEC_CHN VdecChn,
MI_VDEC_VideoStream_t *pstVideoStream, MI_S32 s32MilliSec);
```

- 形参
- 表2-25

| 参数名称           | 描述                                                                                | 输入/输出 |
|----------------|-----------------------------------------------------------------------------------|-------|
| VdecChn        | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                          | 输入    |
| pstVideoStream | 解码码流数据指针。参考 <a href="#">MI_VDEC_VideoStream_t</a> [#318-mi_vdec_videostream_t]定义。 | 输入    |
| s32MilliSec    | 设定推送数据超时时间参数。<br>取值范围：<br>-1：阻塞<br>0：非阻塞<br>正值：超时时间，单位为毫秒                         | 输入    |

- 返回值

表2-26

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或者已销毁   |
| MI_ERR_VDEC_NOT_PERM      | 非法操作，如通道未准备好 |
| MI_ERR_VDEC_BUF_FULL      | 解码后的数据帧缓冲区满  |
| MI_ERR_VDEC_BUSY          | 解码前数据帧缓存区满   |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 支持按帧或按流传送数据。按帧传送时，每次需完整传送一帧数据。暂不支持按流发送。
- 一次发送的1帧数据的大小不能超过码流buffer大小的一半。
- 举例

每次传送完整一帧数据。以帧为单位，如果当前数据帧传送失败，需要重新传送。

```
MI_S32 VdecSendStream(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_CHN VdecChn = 0;
    MI_S32 s32MilliSec = 0;
    MI_VDEC_VideoStream_t stVideoStream;
    MI_VDEC_ChnStat_t stChnStat;

    do{
        //Check if you need stop sending stream
        if(bStop)
        {
            break;
        }

        memset(&stChnStat, 0x0, sizeof(MI_VDEC_ChnStat_t));
        s32Ret = MI_VDEC_GetChnStat(VdecChn, &stChnStat);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_SendStream failed, s32Ret:
0x%x.\n", s32Ret);
            return s32Ret;
        }

        //suggest to check chn status
        if(stChnStat.u32LeftStreamBytes < u32StreamSize)
        {
            continue;
        }

        memset(&stVideoStream, 0x0,
sizeof(MI_VDEC_VideoStream_t));
        stVideoStream.pu8Addr = pu8StreamBuf;
        stVideoStream.u32Len = u32StreamSize;
        stVideoStream.u64PTS = u64StreamPts;
        stVideoStream.bEndOfFrame = TRUE;
        stVideoStream.bEndOfStream = FALSE;
        s32MilliSec = 30; //30ms
```

```

        s32Ret = MI_VDEC_SendStream(VdecChn, &stVideoStream,
s32MilliSec);
        if(MI_SUCCESS != s32Ret)
        {
            printf("MI_VDEC_SendStream failed, s32Ret:
0x%x.\n", s32Ret);
            continue;
        }
    }while(!ShouldStop);

    return MI_SUCCESS;
}

```

## 2.15. MI\_VDEC\_GetUserData

- 功能

获取解码通道的用户数据。
- 语法

```

MI_S32 MI_VDEC_GetUserData(MI_VDEC_CHN
VdecChn,MI_VDEC_UserData_t *pstUserData, MI_S32 s32MilliSec);

```

- 形参

表2-27

| 参数名称        | 描述                                                                             | 输入/输出 |
|-------------|--------------------------------------------------------------------------------|-------|
| VdecChn     | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                       | 输入    |
| s32MilliSec | 获取用户数据超时定义。<br>取值范围：<br>-1：阻塞<br>0：非阻塞<br>正值：推送数据超时时间，以 毫秒为单位                  | 输入    |
| pstUserData | 获取的解码用户数据，参考 <a href="#">MI_VDEC_UserData_t</a><br>[#319-mi_vdec_userdata_t]定义 | 输出    |

- 返回值

表2-28

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或者已销毁   |
| MI_ERR_VDEC_NOT_PERM      | 非法操作，如通道未准备好 |
| MI_ERR_VDEC_BUF_FULL      | 解码后的数据帧缓冲区满  |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 该接口暂不支持。
  - 用户数据是 H.264/H.265 编码时，插入码流 SEI segment 的一段用户私有数据。
  - JPEG 解码通道不支持获取用户数据。
  - [MI\\_VDEC\\_GetUserData](#) [#215-mi\_vdec\_getuserdata] 获取解码用户数据后，需要调用[MI\\_VDEC\\_ReleaseUserData](#) [#216-mi\_vdec\_releaseuserdata]释放获取的数据。
  - 如果获取用户数据不及时，会造成用户数据缓冲区满而出现丢弃用户数据的情况。

---

## 2.16. MI\_VDEC\_ReleaseUserData

- 功能  
释放用户数据。
- 语法

```
MI_S32 MI_VDEC_ReleaseUserData(MI_VDEC_CHN
VdecChn,MI_VDEC_UserData_t * pstUserData);
```

- 形参

表2-29

| 参数名称        | 描述                                                                | 输入/输出 |
|-------------|-------------------------------------------------------------------|-------|
| VdecChn     | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                          | 输入    |
| pstUserData | 解码后的用户数据指针，由 MI_VDEC_UserData_t<br>[#319-mi_vdec_userdata_t] 接口获取 | 输入    |

- 返回值

表2-30

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或者已销毁   |
| MI_ERR_VDEC_NOT_PERM      | 非法操作，如通道未准备好 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

- 注意
  - 该接口暂不支持。
  - 此接口需与 [MI\\_VDEC\\_GetUserData](#) [#215-mi\_vdec\_getuserdata] 配对使用，获取的数据应当在使用完之后立即释放，如果不及时释放，会造成缓冲区满时丢弃用户数据的情况。

## 2.17. MI\_VDEC\_SetDisplayMode

- 功能  
设置显示模式。
- 语法

```
MI_S32 MI_VDEC_SetDisplayMode(MI_VDEC_CHN
VdecChn,MI_VDEC_DisplayMode_e eDisplayMode);
```

- 形参

表2-31

| 参数名称         | 描述                                                                               | 输入/输出 |
|--------------|----------------------------------------------------------------------------------|-------|
| VdecChn      | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                         | 输入    |
| eDisplayMode | 显示模式枚举<br>参考 <a href="#">MI_VDEC_DisplayMode_e</a><br>[#mi_vdec_displaymode_e]定义 | 输入    |

- 返回值

表2-32

| 返回值                       | 描述           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

- 举例

请参见[MI\\_VDEC\\_CreateChn](#) [#24-mi\_vdec\_createchn] 的举例。

---

## 2.18. MI\_VDEC\_GetDisplayMode

- 功能

获取显示模式。
- 语法

```
MI_S32 MI_VDEC_GetDisplayMode(MI_VDEC_CHN
VdecChn, MI_VDEC_DisplayMode_e *peDisplayMode);
```

- 形参

表2-33

| 参数名称          | 描述                                                                                              | 输入/输出 |
|---------------|-------------------------------------------------------------------------------------------------|-------|
| VdecChn       | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                                        | 输入    |
| peDisplayMode | 显示模式枚举指针，参考 <a href="#">MI_VDEC_DisplayMode_e</a> [ <a href="#">#mi_vdec_displaymode_e</a> ]定义。 | 输入    |

- 返回值

表2-34

| 接口**返回值**                 | 含义           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a

---

## 2.19. MI\_VDEC\_SetOutputPortAttr

- 功能

设置解码通道输出端口属性。

- 语法

```
MI_S32 MI_VDEC_SetOutputPortAttr (MI_VDEC_CHN
VdecChn,MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);
```

• 形参

表2-35

| 参数名称              | 描述                                                                                           | 输入/输出 |
|-------------------|----------------------------------------------------------------------------------------------|-------|
| VdecChn           | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                                     | 输入    |
| pstOutputPortAttr | 输出端口属性，数据类型：<br><a href="#">MI_VDEC_OutputPortAttr_t</a> [#320-<br>mi_vdec_outputportattr_t] | 输入    |

• 返回值

表2-36

| 接口返回值                     | 含义            |
|---------------------------|---------------|
| MI_SUCCESS                | 成功。           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁。    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数。         |
| MI_ERR_VDEC_NULL_PTR      | 输入参数空指针       |

• 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

• 注意

- 若通道未创建，则返回错误码MI\_ERR\_VDEC\_CHN\_UNEXIST
- 缩放范围[1/8, 1]。
- 不支持放大。

- 举例

请参见[MI\\_VDEC\\_CreateChn](#) [#24-mi\_vdec\_createchn] 的举例。

## 2.20. MI\_VDEC\_GetOutputPortAttr

- 功能

获取解码通道输出端口属性。

- 语法

```
MI_S32 MI_VDEC_GetOutputPortAttr(MI_VDEC_CHN
VdecChn,MI_VDEC_OutputPortAttr_t *pstOutputPortAttr);
```

- 形参

表2-37

| 参数名称              | 描述                                                                                           | 输入/输出 |
|-------------------|----------------------------------------------------------------------------------------------|-------|
| VdecChn           | 视频解码通道号<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)                                                     | 输入    |
| pstOutputPortAttr | 输出端口属性，数据类型：<br><a href="#">MI_VDEC_OutputPortAttr_t</a> [#320-<br>mi_vdec_outputportattr_t] | 输出    |

- 返回值

表2-38

| 接口**返回值**                 | 含义           |
|---------------------------|--------------|
| MI_SUCCESS                | 成功           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数         |
| MI_ERR_VDEC_NULL_PTR      | 输入参数空指针      |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 若通道未创建，则返回错误码MI\_ERR\_VDEC\_CHN\_UNEXIST

---

## 2.21. MI\_VDEC\_SetOutputPortLayoutMode

- 功能  
设置输出端口Buffer模式。
- 语法

```
MI_S32  
MI_VDEC_SetOutputPortLayoutMode(MI_VDEC_OutbufLayoutMode_e  
eBufTileMode);
```

- 形参

表2-39

| 参数名称         | 描述                                                                                                   | 输入/输出 |
|--------------|------------------------------------------------------------------------------------------------------|-------|
| eBufTileMode | 输出Buffer模式，数据类型：<br><a href="#">MI_VDEC_OutbufLayoutMode_e</a> [#311-<br>mi_vdec_outbuflayoutmode_e] | 输入    |

- 返回值

表2-40

| 接口**返回值**                 | 含义     |
|---------------------------|--------|
| MI_SUCCESS                | 成功     |
| MI_ERR_VDEC_NOT_INIT      | 设备未初始化 |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数   |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 若设备未初始化，则返回错误码MI\_ERR\_VDEC\_NOT\_INIT

- 举例

```
MI_S32 StartVdec(void)
{
    MI_S32 s32Ret = MI_ERR_VDEC_FAILED;
    MI_VDEC_CHN VdecChn = 0;
    MI_VDEC_OutbufLayoutMode_e eBufTileMode =
E_MI_VDEC_OUTBUF_LAYOUT_MAX;
    MI_VDEC_ChnAttr_t stChnAttr;

    memset(&stChnAttr, 0x0, sizeof(MI_VDEC_ChnAttr_t));

    eBufTileMode = E_MI_VDEC_OUTBUF_LAYOUT_AUTO;
    s32Ret = MI_VDEC_SetOutputPortLayoutMode(eBufTileMode) ;
    if(MI_SUCCESS != s32Ret)
    {
```

```

        printf("MI_VDEC_SetOutputPortLayoutMode failed, s32Ret:
0x%x.\n", s32Ret);
        return s32Ret;
    }

    VdecChn = 0;
    stChnAttr.eCodecType      = E_MI_VDEC_CODEC_TYPE_H264;
    stChnAttr.u32PicWidth     = 1920;
    stChnAttr.u32PicHeight    = 1080;
    stChnAttr.eVideoMode      = E_MI_VDEC_VIDEO_MODE_FRAME;
    stChnAttr.u32BufSize      = 1024*1024;
    stChnAttr.eDpbBufMode     =
E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF;
    stChnAttr.stVdecVideoAttr.u32RefFrameNum = 1;
    stChnAttr.u32Priority     = 0;

    s32Ret = MI_VDEC_CreateChn(VdecChn, &stChnAttr);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_VDEC_CreateChn failed, s32Ret: 0x%x.\n",
s32Ret);
        return s32Ret;
    }

    return MI_SUCCESS;
}

```

## 2.22. MI\_VDEC\_GetOutputPortLayoutMode

- 功能

获取输出端口Buffer模式。

- 语法

```

MI_S32
MI_VDEC_GetOutputPortLayoutMode(MI_VDEC_OutbufLayoutMode_e
*peBufTileMode);

```

- 形参

表2-41

| 参数名称         | 描述                                                                                               | 输入/输出 |
|--------------|--------------------------------------------------------------------------------------------------|-------|
| eBufTileMode | 输出Buffer模式，数据类型：<br><a href="#">MI_VDEC_OutbufLayoutMode_e</a> [#311-mi_vdec_outbuflayoutmode_e] | 输入    |

- 返回值

表2-42

| 接口**返回值**            | 含义      |
|----------------------|---------|
| MI_SUCCESS           | 成功      |
| MI_ERR_VDEC_NOT_INIT | 设备未初始化  |
| MI_ERR_VDEC_NULL_PTR | 输入参数空指针 |

- 依赖
  - 头文件：mi\_vdec.h、mi\_common.h
  - 库文件：libmi.a
- 注意
  - 若设备未初始化，则返回错误码MI\_ERR\_VDEC\_NOT\_INIT

---

## 2.23. MI\_VDEC\_SetDestCrop

- 功能

设置解码通道裁剪属性。

- 语法

```
MI_S32 MI_VDEC_SetDestCrop(MI_VDEC_CHN VdecChn,
MI_VDEC_CropCfg_t *pstCropCfg);
```

- 形参

| 参数名称       | 描述                                       | 输入/输出 |
|------------|------------------------------------------|-------|
| VdecChn    | 解码通道号。<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)。 | 输入    |
| pstCropCfg | 解码通道裁剪属性指针，数据类型：<br>MI_VDEC_CropCfg_t。   | 输入    |

- 返回值

| 接口返回值                     | 含义            |
|---------------------------|---------------|
| MI_SUCCESS                | 成功。           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁。    |
| MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数。         |
| MI_ERR_VDEC_NULL_PTR      | 输入参数空指针       |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 裁剪设置的坐标X和宽需要按16对齐，坐标Y和高需要按2对齐；
- 若通道未创建，则返回错误码MI\_ERR\_VDEC\_CHN\_UNEXIST；
- 若裁剪和缩放同时开启，则裁剪宽和高不能大于缩放的宽和高。

- 举例

请参见[MI\\_VDEC\\_CreateChn](#) [#24-mi\_vdec\_createchn]的举例。

## 2.24. MI\_VDEC\_GetDestCrop

- 功能

获取解码通道裁剪属性。

- 语法

```
MI_S32 MI_VDEC_GetDestCrop(MI_VDEC_CHN VdecChn,
MI_VDEC_CropCfg_t *pstCropCfg)
```

- 形参

| 参数名称       | 描述                                       | 输入/输出 |
|------------|------------------------------------------|-------|
| VdecChn    | 解码通道号。<br>取值范围：[0, MI_VDEC_MAX_CHN_NUM)。 | 输入    |
| pstCropCfg | 解码通道裁剪属性指针，数据类型:<br>MI_VDEC_CropCfg_t。   | 输出    |

- 返回值

| 接口返回值                     | 含义            |
|---------------------------|---------------|
| MI_SUCCESS                | 成功。           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁。    |
| MI_ERR_VDEC_NULL_PTR      | 输入参数空指针       |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a

- 注意

- 若通道未创建，则返回错误码MI\_ERR\_VDEC\_CHN\_UNEXIST；
-

## 2.25. MI\_VDEC\_SetChnErrHandlePolicy

- 功能

设置解码通道错误宏块的数据帧的输出策略。

- 语法

```
MI_S32 MI_VDEC_SetChnErrHandlePolicy(MI_VDEC_CHN VdecChn,
MI_VDEC_ErrHandlePolicy_t *pstErrHandlePolicy)
```

- 形参

| 参数名称                           | 描述                                             | 输入/输出 |
|--------------------------------|------------------------------------------------|-------|
| VdecChn                        | 解码通道号。                                         |       |
| 取值范围：[0, MI_VDEC_MAX_CHN_NUM)。 | 输入                                             |       |
| pstErrHandlePolicy             | 设置错误宏块的数据帧的输出策略，参考MI_VDEC_ErrHandlePolicy_t定义。 |       |
| 输入                             |                                                |       |

- 返回值

| 接口返回值                     | 含义            |
|---------------------------|---------------|
| MI_SUCCESS                | 成功。           |
| MI_ERR_VDEC_INVALID_CHNID | 通道 ID 超出合法范围。 |
| MI_ERR_VDEC_CHN_UNEXIST   | 通道未创建或已销毁。    |
| MI_ERR_VDEC_NULL_PTR      | 输入参数空指针       |

- 依赖

- 头文件：mi\_vdec.h、mi\_common.h
- 库文件：libmi.a
- 注意
  - 若通道未创建，则返回错误码MI\_ERR\_VDEC\_CHN\_UNEXIST；

### 3. 数据类型

#### 3.1. 数据类型定义

视频解码相关数据类型、数据结构定义如下表。

表3-1

| 数据结构                                               | 说明            |
|----------------------------------------------------|---------------|
| MI_VDEC_CodecType_e [#32-mi_vdec_codectype_e]      | 定义解码类型        |
| MI_VDEC_DPB_BufMode_e [#33-mi_vdec_dpb_bufmode_e]  | 定义Inplace内存模式 |
| MI_VDEC_JpegFormat_e [#34-mi_vdec_jpegformat_e]    | 定义Jpeg 图片数据格式 |
| MI_VDEC_VideoMode_e [#35-mi_vdec_videomode_e]      | 定义码流发送方式枚举    |
| MI_VDEC_ErrCode_e [#36-mi_vdec_errcode_e]          | 定义码流发送方式      |
| MI_VDEC_DecodeMode_e [#37-mi_vdec_decodemode_e]    | 定义码流解码方式      |
| MI_VDEC_OutputOrder_e [#38-mi_vdec_outputorder_e]  | 定义码流发送方式      |
| MI_VDEC_VideoFormat_e [#39-mi_vdec_videoformat_e]  | 定义解码图像数据格式枚举  |
| MI_VDEC_DisplayMode_e [#310-mi_vdec_displaymode_e] | 定义显示模式枚举      |

|                                                                              |                      |
|------------------------------------------------------------------------------|----------------------|
| <a href="#">MI_VDEC_OutbufLayoutMode_e</a> [#311-mi_vdec_outbuflayoutmode_e] | 定义输出buffer模式枚举       |
| <a href="#">MI_VDEC_InitParam_t</a> [#312-mi_vdec_initparam_t]               | 定义解码设备初始化参数结构体       |
| <a href="#">MI_VDEC_ChnAttr_t</a> [#313-mi_vdec_chnattr_t]                   | 定义视频解码通道属性           |
| <a href="#">MI_VDEC_JpegAttr_t</a> [#314-mi_vdec_jpegattr_t]                 | 定义JPEG视频解码通道属性结构体    |
| <a href="#">MI_VDEC_VideoAttr_t</a> [#315-mi_vdec_videoattr_t]               | 定义视频解码通道属性           |
| <a href="#">MI_VDEC_ChnStat_t</a> [#316-mi_vdec_chnstat_t]                   | 定义通道状态结构体            |
| <a href="#">MI_VDEC_ChnParam_t</a> [#317-mi_vdec_chnparam_t]                 | 定义解码通道参数结构体          |
| <a href="#">MI_VDEC_VideoStream_t</a> [#318-mi_vdec_videostream_t]           | 定义推送码流结构体            |
| <a href="#">MI_VDEC_UserData_t</a> [#319-mi_vdec_userdata_t]                 | 定义用户数据结构体            |
| <a href="#">MI_VDEC_OutputPortAttr_t</a> [#320-mi_vdec_outputportattr_t]     | 定义输出端口属性结构体          |
| <a href="#">MI_VDEC_ErrHandlePolicy_t</a> [#321-mi_vdec_errhandlepolicy_t]   | 控制有错误宏块的数据帧是否需要输出显示。 |
| <a href="#">MI_VDEC_CropCfg_t</a> [#322-mi_vdec_cropcfg_t]                   | 定义裁剪属性结构体            |

## 3.2. MI\_VDEC\_CodecType\_e

- 说明  
定义解码类型。
- 定义

```
typedef enum
{
```

```

E_MI_VDEC_CODEC_TYPE_H264 = 0x0,          /* H264 */

E_MI_VDEC_CODEC_TYPE_H265,          /* H265 */

E_MI_VDEC_CODEC_TYPE_JPEG,          /* JPEG */

E_MI_VDEC_CODEC_TYPE_MAX

} MI_VDEC_CodecType_e;

```

### 3.3. MI\_VDEC\_DPB\_BufMode\_e

- 说明

定义DPB buffer模式。

- 定义

```

typedef enum

{

    E_MI_VDEC_DPB_MODE_NORMAL=0,

    E_MI_VDEC_DPB_MODE_INPLACE_ONE_BUF=1,

    E_MI_VDEC_DPB_MODE_INPLACE_TWO_BUF=2,

    E_MI_VDEC_DPB_MODE_MAX

} MI_VDEC_DPB_BufMode_e;

```

- 成员

表3-2

| 成员名称                               | 描述                    |
|------------------------------------|-----------------------|
| E_MI_VDEC_DPB_MODE_NORMAL          | 普通模式                  |
| E_MI_VDEC_DBP_MODE_INPLACE_ONE_BUF | Inplace one buffer模式。 |
| E_MI_VDEC_DBP_MODE_INPLACE_TWO_BUF | Inplace two buffer模式  |

---

### 3.4. MI\_VDEC\_JpegFormat\_e

- 说明

Jpeg 图片的存储格式。

- 定义

```
typedef enum
{
    E_MI_VDEC_JPEG_FORMAT_YCBCR400 = 0x0,    /*YUV400*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR420, /*YUV420*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR422, /*YUV 422*/
    E_MI_VDEC_JPEG_FORMAT_YCBCR444, /*YUV 444*/
    E_MI_VDEC_JPEG_FORMAT_MAX

} MI_VDEC_JpegFormat_e;
```

- 注意事项

该参数暂不支持设置。

---

### 3.5. MI\_VDEC\_VideoMode\_e

- 说明

定义码流发送方式。

- 定义

```
typedef enum
{
    E_MI_VDEC_VIDEO_MODE_STREAM = 0x0,
    E_MI_VDEC_VIDEO_MODE_FRAME,
    E_MI_VDEC_VIDEO_MODE_MAX
} MI_VDEC_VideoMode_e;
```

- 成员

表3-3

| 成员名称                   | 描述             |
|------------------------|----------------|
| E_MI_VIDEO_MODE_STREAM | 按流方式发送码流       |
| E_MI_VIDEO_MODE_FRAME  | 按帧方式发送码流,以帧为单位 |

3.6. MI\_VDEC\_ErrCode\_e

- 说明

定义码流发送方式。

- 定义

```
typedef enum
{
    E_MI_VDEC_ERR_CODE_UNKNOW = 0x0,
    E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS,
```

```

E_MI_VDEC_ERR_CODE_FRMRATE_UNSupport,

E_MI_VDEC_ERR_CODE_DEC_TIMEOUT,

E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY,

E_MI_VDEC_ERR_CODE_CODEEC_TYPE_UNSupport,

E_MI_VDEC_ERR_CODE_ERR_SPS_UNSupport,

E_MI_VDEC_ERR_CODE_ERR_PPS_UNSupport,

E_MI_VDEC_ERR_CODE_REF_LIST_ERR,

E_MI_VDEC_ERR_CODE_MAX

} MI_VDEC_ErrCode_e;

```

- 成员

表3-4

| 成员名称                                     | 描述                  |
|------------------------------------------|---------------------|
| E_MI_VDEC_ERR_CODE_UNKNOW                | 未知错误。               |
| E_MI_VDEC_ERR_CODE_ILLEGAL_ACCESS        | 不可访问，例如:未初始化或硬件出问题。 |
| E_MI_VDEC_ERR_CODE_FRMRATE_UNSupport     | 帧率不支持               |
| E_MI_VDEC_ERR_CODE_DEC_TIMEOUT           | 解帧超时                |
| E_MI_VDEC_ERR_CODE_OUT_OF_MEMORY         | 内存不足                |
| E_MI_VDEC_ERR_CODE_CODEEC_TYPE_UNSupport | 解码类型不支持             |
| E_MI_VDEC_ERR_CODE_ERR_SPS_UNSupport     | 不支持的SPS或SPS出错       |
| E_MI_VDEC_ERR_CODE_ERR_PPS_UNSupport     | 不支持的PPS或PPS出错       |
| E_MI_VDEC_ERR_CODE_REF_LIST_ERR          | 参考帧列表出错             |

---

### 3.7. MI\_VDEC\_DecodeMode\_e

- 说明

定义码流解码方式。

- 定义

```
typedef enum
{
    E_MI_VDEC_DECODE_MODE_ALL = 0x0,
    E_MI_VDEC_DECODE_MODE_I,
    E_MI_VDEC_DECODE_MODE_IP,
    E_MI_VDEC_DECODE_MODE_MAX
} MI_VDEC_DecodeMode_e;
```

- 成员

表3-5

| 成员名称                      | 描述         |
|---------------------------|------------|
| E_MI_VDEC_DECODE_MODE_ALL | 解码IPB数据帧模式 |
| E_MI_VDEC_DECODE_MODE_I   | 只解I帧模式     |
| E_MI_VDEC_DECODE_MODE_IP  | 只解IP帧，跳过B帧 |

---

### 3.8. MI\_VDEC\_OutputOrder\_e

- 说明

定义码流发送方式。

- 定义

typedef enum

{

```
E_MI_VDEC_OUTPUT_ORDER_DISPLAY = 0x0,

E_MI_VDEC_OUTPUT_ORDER_DECODE,

E_MI_VDEC_OUTPUT_ORDER_MAX,
```

} MI\_VDEC\_OutputOrder\_e;

- 成员

表3-6

| 成员名称                           | 描述          |
|--------------------------------|-------------|
| E_MI_VDEC_OUTPUT_ORDER_DISPLAY | 按显示顺序输出数据帧  |
| E_MI_VDEC_OUTPUT_ORDER_DECODE  | 按解帧顺序列输出数据帧 |

### 3.9. MI\_VDEC\_VideoFormat\_e

- 说明

定义解码图像数据格式枚举。

- 定义

```
typedef enum

{

    E_MI_VDEC_VIDEO_FORMAT_TILE = 0x0,

    E_MI_VDEC_VIDEO_FORMAT_REDUCE,

    E_MI_VDEC_VIDEO_FORMAT_MAX

} MI_VDEC_VideoFormat_e;
```

- 成员

表3-7

| 成员名称                          | 描述                  |
|-------------------------------|---------------------|
| E_MI_VDEC_VIDEO_FORMAT_TILE   | TILE数据格式。           |
| E_MI_VDEC_VIDEO_FORMAT_REDUCE | 数据帧压缩模式，减少数据帧内存使用量。 |

- 注意事项
  - 当前数据类型不支持上层设定，只能返回支持数据类型。

### 3.10. MI\_VDEC\_DisplayMode\_e

- 说明

定义显示模式枚举。

- 定义

```
typedef enum
{
    E_MI_VDEC_DISPLAY_MODE_PREVIEW = 0x0,
    E_MI_VDEC_DISPLAY_MODE_PLAYBACK,
    E_MI_VDEC_DISPLAY_MODE_MAX,
} MI_VDEC_DisplayMode_e;
```

- 成员

表3-8

| 成员名称                       | 描述             |
|----------------------------|----------------|
| E_MI_DISPLAY_MODE_PREVIEW  | 预览模式。不参考PTS输出  |
| E_MI_DISPLAY_MODE_PLAYBACK | 回放模式。参考PTS数值输出 |

### 3.11. MI\_VDEC\_OutbufLayoutMode\_e

- 说明  
定义输出buffer模式枚举。
- 定义

```
typedef enum
{
    E_MI_VDEC_OUTBUF_LAYOUT_AUTO = 0x0,
    E_MI_VDEC_OUTBUF_LAYOUT_LINEAR,
    E_MI_VDEC_OUTBUF_LAYOUT_TILE,
    E_MI_VDEC_OUTBUF_LAYOUT_MAX
} MI_VDEC_OutbufLayoutMode_e;
```

- 成员

表3-9

| 成员名称                           | 描述                |
|--------------------------------|-------------------|
| E_MI_VDEC_OUTBUF_LAYOUT_AUTO   | 输出buffer模式自适应     |
| E_MI_VDEC_OUTBUF_LAYOUT_LINEAR | 输出buffer为LINEAR模式 |
| E_MI_VDEC_OUTBUF_LAYOUT_TILE   | 输出buffer为TILE模式   |

---

## 3.12. MI\_VDEC\_InitParam\_t

- 说明

定义解码设备初始化参数结构体。

- 定义

```
typedef struct MI_VDEC_InitParam_s
{
    MI_BOOL bDisableLowLatency;
} MI_VDEC_InitParam_t;
```

- 成员

表3-10

| 成员名称               | 描述                                    |
|--------------------|---------------------------------------|
| bDisableLowLatency | 开启/关闭低延时模式，默认为FALSE。<br>参数类型: MI_BOOL |

- 注意事项

- 若bDisableLowLatency设为TRUE，表示关闭低延时；这种情况适用于解码B帧，用于对B帧的输出做reorder，输出画面会有延时。另外，做reorder操作，也会增加内存使用量。
- 默认开启了低延时模式，即bDisableLowLatency为FALSE。如果场景不涉及延时模式，则无需调用[MI\\_VDEC\\_InitDev](#) [#22-mi\_vdec\_initdev]。

---

## 3.13. MI\_VDEC\_ChnAttr\_t

- 说明

定义解码通道属性结构体。

- 定义

```
typedef struct MI_VDEC_ChnAttr_s
{
    MI_VDEC_CodecType_e eCodecType;

    MI_U32  u32BufSize

    MI_U32  u32Priority

    MI_U32  u32PicWidth

    MI_U32  u32PicHeight

    MI_VDEC_VideoMode_e eVideoMode

    MI_VDEC_DPB_BufMode_e eDpbBufMode;

    union
    {
        MI_VDEC_JpegAttr_t stVdecJpegAttr;

        MI_VDEC_VideoAttr_t stVdecVideoAttr;

    };

} MI_VDEC_ChnAttr_t;
```

- 成员

表3-11

| 成员名称            | 描述                                                                              |
|-----------------|---------------------------------------------------------------------------------|
| eCodecType      | 解码类型枚举值<br>参数类型: <a href="#">MI_VDEC_CodecType_e</a> [#32-mi_vdec_codec_type_e] |
| u32BufSize      | 码流缓存的大小                                                                         |
| u32Priority     | 通道优先级，取值范围为 1 ~ 255，值越大优先级越高                                                    |
| u32PicWidth     | 通道支持的解码图像最大宽（以像素为单位）                                                            |
| u32PicHeight    | 通道支持的解码图像最大高（以像素为单位）                                                            |
| eVideoMode      | 码流传送方式                                                                          |
| eDpbBufMode     | DPB buffer 模式                                                                   |
| stVdecJpegAttr  | JPEG 通道的相关属性，暂不支持设置                                                             |
| stVdecVideoAttr | 除 JPEG 以外其它所支持类型的通道的相关属性                                                        |

- 注意:
  - 当前暂不支持此功能。只支持按帧。
  - eDpbBufMode=1，只适用于一个参考帧的码流；eDpbBufMode=2，适用于两个参考帧的码流；如果超过两个参考帧，则需要设置为 eDpbBufMode=0
  - 当设置了eDpbBufMode为E\_MI\_VDEC\_DBP\_MODE\_INPLACE\_ONE\_BUF 或者E\_MI\_VDEC\_DBP\_MODE\_INPLACE\_TWO\_BUF时，u32RefFrameNum的值将无效；
  - 当eInplaceMode为E\_MI\_VDEC\_DPB\_MODE\_NORMAL时，需要设置 u32RefFrameNum的值，用于限制最大参考帧个数，避免分配过多的 frame buffer；若u32RefFrameNum的值小于当前解码所需的参考帧个数，则可能不解码或者解码花屏。

### 3.14. MI\_VDEC\_JpegAttr\_t

- 说明

定义JPEG 解码通道属性。

- 定义

```
typedef struct MI_VDEC_JpegAttr_s
{
    MI_VDEC_JpegFormat_e eJpegFormat;

}MI_VDEC_JpegAttr_t;
```

- 成员

表3-12

| 成员名称        | 描述                                           |
|-------------|----------------------------------------------|
| eJpegFormat | JPEG图片的存储格式<br>取值范围是[0,E_MI_JPEG_FORMAT_MAX) |

- 注意事项

- 暂不支持设置该参数。

### 3.15. MI\_VDEC\_VideoAttr\_t

- 说明

定义视频解码通道属性。

- 定义

```
typedef struct MI_VDEC_VideoAttr_s
{
    MI_U32 u32RefFrameNum;

    MI_VDEC_ErrHandlePolicy_t stErrHandlePolicy;
```

```
}MI_VDEC_VideoAttr_t;
```

• 成员

表3-13

| 成员名称              | 描述                                                                                                |
|-------------------|---------------------------------------------------------------------------------------------------|
| u32RefFrameNum    | 参考帧的数目<br>取值范围：[0, 16]，以帧为单位<br>参考帧的数目决定解码时需要的参考帧个数，影响内存使用量，根据实际情况设置合适的值。测试码流：推荐设为 2              |
| stErrHandlePolicy | 设置错误宏块的数据帧的输出策略，参考 <a href="#">MI_VDEC_ErrHandlePolicy_t</a> [#321-mi_vdec_errhandlepolicy_t] 定义。 |

• 注意事项

- 当设置了eDpbBufMode为E\_MI\_VDEC\_DBP\_MODE\_INPLACE\_ONE\_BUF或者E\_MI\_VDEC\_DBP\_MODE\_INPLACE\_TWO\_BUF时，u32RefFrameNum的值将无效；
- 当eInplaceMode为E\_MI\_VDEC\_DPB\_MODE\_NORMAL时，需要设置u32RefFrameNum的值，用于限制最大参考帧个数，避免分配过多的frame buffer；若u32RefFrameNum的值小于当前解码所需的参考帧个数，则可能不解码或者解码花屏。

3.16. MI\_VDEC\_ChnStat\_t

• 说明

定义通道状态结构体。

• 定义

```
typedef MI_VDEC_ChnStat_s  
  
{
```

```

MI_VDEC_CodecType_e eCodecType;

MI_U32  u32LeftStreamBytes;

MI_U32  u32LeftStreamFrames;

MI_U32  u32LeftPics;

MI_BOOL bChnStart;

MI_U32  u32RecvStreamFrames;

MI_U32  u32DecodeStreamFrames;

MI_VDEC_ErrCode_e eErrCode;

} MI_VDEC_ChnStat_t;

```

- 成员

表3-14

| 成员名称                  | 描述                                           |
|-----------------------|----------------------------------------------|
| eCodecType            | 解码类型                                         |
| u32LeftStreamBytes    | 码流 buffer 中待解码的 byte 数。                      |
| u32LeftStreamFrames   | 码流 buffer 中待解码的帧数。<br>-1 表示无效。<br>仅按帧发送时有效。  |
| u32LeftPics           | 图像 buffer 中剩余的 pic 数目。                       |
| bChnStart             | 解码器是否已经启动接收码流。                               |
| u32RecvStreamFrames   | 码流 buffer 中已接收码流帧数。<br>-1 表示无效。<br>仅按帧发送时有效。 |
| u32DecodeStreamFrames | 码流 buffer 中已解码帧数。                            |
| eErrCode              | 解码错误码信息。                                     |

### 3.17. MI\_VDEC\_ChnParam\_t

- 说明  
定义解码通道参数。
- 定义

```
typedef struct MI_VDEC_ChnParam_s
{
    MI_VDEC_DecodeMode_e eDecMode;

    MI_VDEC_OutputOrder_e eOutputOrder;

    MI_VDEC_VideoFormat_e eVideoFormat;
} MI_VDEC_ChnParam_t;
```

- 成员

表3-15

| 成员名称         | 描述                                                                                                                                        |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| eDecMode     | 解码模式，参考 <a href="#">MI_VDEC_DecodeMode_e</a> [#37-mi_vdec_decodemode_e]定义。默认为 E_MI_VDEC_DECODE_MODE_IP。E_MI_VDEC_DECODE_MODE_ALL模式需要足够内存。 |
| eOutputOrder | 参考 <a href="#">MI_VDEC_OutputOrder_e</a> [#38-mi_vdec_outputorder_e]定义，默认为 E_MI_VDEC_OUTPUT_ORDER_DECODE 按解码顺序输出数制帧                       |
| eVideoFormat | 参考 <a href="#">MI_VDEC_VideoFormat_e</a> [#39-mi_vdec_videoformat_e]定义。                                                                   |

### 3.18. MI\_VDEC\_VideoStream\_t

- 说明

定义视频解码的码流结构体。

- 定义

```
typedef struct MI_VDEC_VideoStream_s
{
    MI_U8*   pu8Addr;

    MI_U32  u32Len;

    MI_U64  u64PTS;

    MI_BOOL bEndOfFrame;

    MI_BOOL bEndOfStream;

}MI_VDEC_VideoStream_t;
```

- 成员

表3-16

| 成员名称         | 描述                                           |
|--------------|----------------------------------------------|
| pu8Addr      | 码流包的地址。                                      |
| u32Len       | 码流包的长度，以字节为单位。                               |
| u64PTS       | 码流包的时间戳，以毫秒为单位。                              |
| bEndOfFrame  | 当前帧是否结束，预留，当前只支持帧模式下按帧传送。                    |
| bEndOfStream | 是否发完所有码流。当所有码流数据帧传送完毕时，bEndOfStream值需置为TRUE。 |

- 注意事项

- 支持按帧或按流传送数据。按帧传送时，每次需完整传送一帧数据。暂不支持按流发送。

- 码流帧数据附带PTS时，解码后输出数据输出相同PTS。当PTS=-1时，不参考系统时钟输出数据帧。

### 3.19. MI\_VDEC\_UserData\_t

- 说明

定义用户数据结构体。

- 定义

```
typedef struct MI_VDEC_UserData_s
{
    MI_U8* pu8Addr;

    MI_U32 u32Len;

    MI_BOOL bValid;
} MI_VDEC_UserData_t;
```

- 成员

表3-17

| 成员名称    | 描述                                                     |
|---------|--------------------------------------------------------|
| pu8Addr | 用户数据的虚拟地址。                                             |
| u32Len  | 用户数据的长度。以 byte 为单位。                                    |
| bValid  | 当前数据的有效标识。取值范围：{TRUE, FALSE}。<br>TRUE：有效。<br>FALSE：无效。 |

### 3.20. MI\_VDEC\_OutputPortAttr\_t

- 说明

定义输出端口属性结构体。

- 定义

```
typedef struct MI_VDEC_OutputPortAttr_s
{
    MI_U16 u16Width; // Width of target image
    MI_U16 u16Height; // Height of target image
}MI_VDEC_OutputPortAttr_t;
```

- 成员

表3-18

| 成员名称      | 描述    |
|-----------|-------|
| u16Width  | 输出图像宽 |
| u16Height | 输出图像高 |

- 注意事项

- 缩放范围[1/8, 1]。
- 不支持放大。

### 3.21. MI\_VDEC\_ErrHandlePolicy\_t

- 说明

控制有错误宏块的数据帧是否需要输出显示。

- 定义

```
typedef struct MI_VDEC_ErrHandlePolicy_s
{
```

```

    MI_BOOL bUseCusPolicy;           // FALSE: use default;
    TRUE: use customized value

    MI_U8 u8ErrMBPercentThreshold;

} MI_VDEC_ErrHandlePolicy_t;

```

- 成员

| 成员名称                    | 描述                                           |
|-------------------------|----------------------------------------------|
| bUseCusPolicy           | 是否需要客制化控制错误帧输出策略。                            |
| u8ErrMBPercentThreshold | 设置控制不输出数据帧时，错误宏块数占整张数据帧宏块数的比率值，取值范围[0, 100]。 |

- 注意事项
- bUseCusPolicy 值为FALSE时代表不做客制化，u8ErrMBPercentThreshold为系统内置默认值30;值为TRUE时，使用客制化定义的u8ErrMBPercentThreshold数值。
- u8ErrMBPercentThreshold取值范围[0, 100]。例如数值为10时，代表错误宏块个数占整个数据帧宏块个数低于10%时，当前数据帧输出显示，否则(大于等于10%)不显示。其中数值为0时，代表只要有错误宏块都将不输出显示；数值为100时，代表不管有多少错误宏块当前数据帧都将会显示。
- u8ErrMBPercentThreshold数值是以宏块为基本单位计算比率值。

## 3.22. MI\_VDEC\_CropCfg\_t

- 说明  
定义裁剪属性结构体。
- 定义

```

typedef struct MI_VDEC_CropCfg_s
{

```

```
MI_BOOL bEnable; /* Crop region enable */

MI_SYS_WindowRect_t stRect; /* Crop region */

} MI_VDEC_CropCfg_t;
```

• 成员

| 成员名称    | 描述                                         |
|---------|--------------------------------------------|
| bEnable | 开启/关闭裁剪功能。                                 |
| stRect  | 裁剪属性。结构体MI_SYS_WindowRect_t参考mi_sys数据类型定义。 |

• 注意事项

- 取值范围：u16X:[0, 4096-16]，u16Y:[0, 4096-2]，u16Width:[16, 4096]，u16Height:[2, 4096]；
- 对齐要求：u16X / u16Width需按16对齐，u16Y和u16Height需按2对齐。

4. 错误码

表4-1 VDEC API错误码

| 错误代码       | 宏定义                       | 描述                |
|------------|---------------------------|-------------------|
| 0xA0082001 | MI_ERR_VDEC_INVALID_DEVID | 设备ID超出合法范围        |
| 0xA0082002 | MI_ERR_VDEC_INVALID_CHNID | 通道ID超出合法范围        |
| 0xA0082003 | MI_ERR_VDEC_ILLEGAL_PARAM | 非法参数或输入参数超过通道解码能力 |
| 0xA0082004 | MI_ERR_VDEC_CHN_EXIST     | 试图创建已经存在的通道       |
| 0xA0082005 | MI_ERR_VDEC_CHN_UNEXIST   | 通道不存在             |
| 错误代码       | 宏定义                       | 描述                |
| 0xA0082006 | MI_ERR_VDEC_NULL_PTR      | 输入参数为空指针          |

|            |                           |                       |
|------------|---------------------------|-----------------------|
| 0xA0082006 | MI_ERR_VDEC_NULL_PTR      | 输入参数为全0值              |
| 0xA0082007 | MI_ERR_VDEC_NOT_CONFIG    | 使用前未配置                |
| 0xA0082008 | MI_ERR_VDEC_NOT_SUPPORT   | 该操作或者功能不支持            |
| 0xA0082009 | MI_ERR_VDEC_NOT_PERM      | 非法操作                  |
| 0xA008200C | MI_ERR_VDEC_NOMEM         | 分配内存失败，如系统内存不足        |
| 0xA008200D | MI_ERR_VDEC_NOBUF         | 分配缓存失败，如申请的数据缓冲区太大    |
| 0xA008200E | MI_ERR_VDEC_BUF_EMPTY     | 缓冲区中无数据               |
| 0xA008200F | MI_ERR_VDEC_BUF_FULL      | 缓冲区中数据满               |
| 0xA0082010 | MI_ERR_VDEC_SYS_NOTREADY  | 系统没有初始化或者相关 依赖的模块没有加载 |
| 0xA0082011 | MI_ERR_VDEC_BADADDR       | 地址错误                  |
| 0xA0082012 | MI_ERR_VDEC_BUSY          | 系统忙                   |
| 0xA0082013 | MI_ERR_VDEC_CHN_NOT_START | 通道未使能                 |
| 0xA0082014 | MI_ERR_VDEC_CHN_NOT_STOP  | 通道未禁用                 |
| 0xA0082015 | MI_ERR_VDEC_NOT_INIT      | 模块没有成功加载              |
| 0xA0082016 | MI_ERR_VDEC_INITED        | 模块已成功加载               |
| 0xA008201F | MI_ERR_VDEC_FAILED        | 失败                    |

## 5. PROCFS介绍

# 5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_vdec/mi_vdec0

=====Private Vdec0 Info=====
-----CHN ATTR Info-----
ChnID  CodecType  Width  Height  BufSize  Priority  VideoMode  JpegFormat  RefFrmNum
0      0          720    576    1048576  0         1          4          4
1      0          720    576    1048576  0         1          4          4
2      0          720    576    1048576  0         1          4          4
3      0          720    576    1048576  0         1          4          4

-----CHN PARAM Info-----
ChnID  DecMode  OutputOrder  VideoFormat  DisplayMode
0      0         0            0            0
1      0         0            0            0
2      0         0            0            0
3      0         0            0            0

-----CHN STATE-----
ChnID  bStart  CodecType  LeftStrmBytes  LeftStrmFrm  LeftPic  RecvStrmFrm  DecStrmFrm
0      1        0           0              0            396       396         396
1      1        0       116265        1            396       397         396
2      1        0           0              0            396       396         396
3      1        0           0              0            397       398         398

ChnID  ErrCnt  DropCnt  SkipCnt
0      0        0        0
1      0        0        0
2      0        0        0
3      1        0        0
```

- 调试信息分析

打印分为两部分，使用Private Vdec0 Info分隔开。上半部分为common信息，下半部分为vdec模块信息。主要记录了解码通道的使用情况及配置属性，可用于检查属性配置和当前通道的工作状态，便于debug。

- 参数说明

| 参数            |           | 描述                                      |
|---------------|-----------|-----------------------------------------|
| CHN ATTR Info | ChnID     | 通道号。                                    |
|               | CodecType | 解码协议类型<br>0：H264;<br>1：H265;<br>2：JPEG。 |
|               | Width     | 解码图像最大宽度。                               |
|               | Height    | 解码图像最大高度。                               |

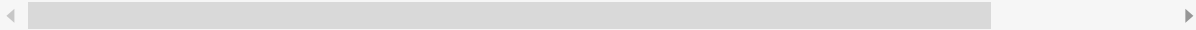
|                   |             |                                                                         |
|-------------------|-------------|-------------------------------------------------------------------------|
|                   | BufSize     | VDEC码流buffer大小，单位：<br>byte。                                             |
|                   | Priority    | 解码通道优先级。                                                                |
|                   | VideoMode   | 发送码流方式。<br>0：按流发送；<br>1：按帧发送。                                           |
|                   | JpegFormat  | Jpeg图片的存储格式<br>0：YCbCr400；<br>1：YCbCr420；<br>2：YCbCr422；<br>3：YCbCr444。 |
|                   | RefFrmNum   | 最大参考帧个数，JPEG通道无<br>效。                                                   |
| CHN<br>PARAM Info | ChnID       | 通道号。                                                                    |
|                   | DecMode     | 解码模式。<br>0：解码IPB数据帧模式；<br>1：只解I帧模式；<br>2：只解IP帧，跳过B帧。                    |
|                   | OutputOrder | 解码图像输出顺序。<br>0：按显示顺序输出数据帧；<br>1：按解帧顺序输出数据帧。                             |
|                   | VideoFormat | 解码图像数据格式。<br>0：TILE数据格式；<br>1：数据帧压缩模式，减少数据帧<br>内存使用量。                   |
|                   | DisplayMode | 显示模式。<br>0：预览模式，不参考PTS输出。<br>1：回放模式，参考PTS数值输<br>出。                      |
| CHN STATE         | ChnID       | 通道号。                                                                    |
|                   | bStart      | 解码器是否启动接收码流。                                                            |

|               |                                              |
|---------------|----------------------------------------------|
| CodecType     | 解码协议类型<br>0 : H264;<br>1 : H265;<br>2 : JPEG |
| LeftStrmBytes | 码流buffer中待解码的byte数。                          |
| LeftStrmFrm   | 码流buffer中待解码的帧数。-1 表示无效。仅按帧发送时有效。            |
| LeftPics      | 图像buffer中剩余的pic数目。                           |
| RecvStrmFrm   | 码流buffer中已接收码流帧数。-1 表示无效。仅按帧发送时有效。           |
| DecStrmFrm    | 码流buffer中已解码帧数。                              |

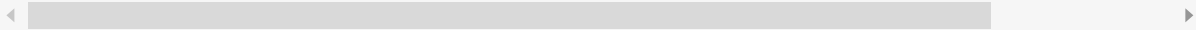
## 5.2. echo

| 功能   | 设置解码模式                                                                               |
|------|--------------------------------------------------------------------------------------|
| 命令   | <code>echo setDecMode [ChnID] [ModID] &gt; /proc/mi_modules/mi_vdec/mi_vdec0</code>  |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[ModID] 解码模式: 0->H264; 1->H265; 2->JPEG                        |
| 举例   | <code>echo setDecMode 0 0 &gt; /proc/mi_modules/mi_vdec/mi_vdec0 //</code> 通道0，解码模式为 |

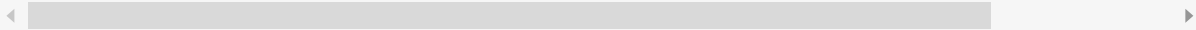
|      |                                                                         |
|------|-------------------------------------------------------------------------|
| 功能   | 设置解码图像输出顺序                                                              |
| 命令   | echo setOutputOrder [ChnID] [ModID] > /proc/mi_modules/mi_vdec/mi_vdec0 |
| 参数说明 | ChnID 通道号 [0~32]<br>ModID 0: 按显示顺序输出数据帧; 1: 按解码顺序输出数据帧                  |
| 举例   | echo setOutputOrder 0 0 > /proc/mi_modules/mi_vdec/mi_vdec0 //通道0，按显示顺  |



|      |                                                                         |
|------|-------------------------------------------------------------------------|
| 功能   | 设置解码图像数据格式，暂不支持设置                                                       |
| 命令   | echo setVideoFormat [ChnID] [ModID] > /proc/mi_modules/mi_vdec/mi_vdec0 |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[ModID] 解码数据格式<br>0: TILE格式 1: 数据帧压缩模式，减少数据帧内存使用量 |
| 举例   | echo setVideoFormat 0 0 > /proc/mi_modules/mi_vdec/mi_vdec0 //通道0，TILE格 |



|      |                                                                         |
|------|-------------------------------------------------------------------------|
| 功能   | 设置显示模式                                                                  |
| 命令   | echo setDisplayMode [ChnID] [ModID] > /proc/mi_modules/mi_vdec/mi_vdec0 |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[ModID] 0：预览模式，不参考PTS输出。 1：回放模式，参考PTS数值输出。        |
| 举例   | echo setDisplayMode 0 0 > /proc/mi_modules/mi_vdec/mi_vdec0 //通道0 预览模式  |



|      |                                                                                                                                                                                                                                                                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 打开/关闭统计打印码流输入的时间间隔，每隔一秒打印一次                                                                                                                                                                                                                                                                                                                            |
| 命令   | echo statInputTimeIntvl [ChnID] [Status] >/proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                                                                                                                                                                            |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[Status] on 打开 off 关闭                                                                                                                                                                                                                                                                                                            |
| 举例   | echo statInputTimeIntvl 0 on >/proc/mi_modules/mi_vdec/mi_vdec0 [MI VDEC PRO<br>ChnID: 0, CurTime: 5959100469, TotalFrmCnt: 19, AvgTimeIntvl: 49856, MaxTimeInt<br>MinTimeIntvl: 47252<br>[Input ChnID] 通道号<br>[CurTime] 当前时间<br>[TotalFrmCnt] 输入帧次数<br>[AvgTimeIntvl] 每秒内所有帧输入平均时间间隔<br>[MaxTimeIntvl] 每秒内两帧输入最大时间间隔<br>[MinTimeIntvl] 每秒内两帧输入最小时间间隔。 |



|      |                                                                                                                                                                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 打开/关闭统计打印图像输出的时间间隔                                                                                                                                                                                                                                                                                                                                                      |
| 命令   | echo statOutputTimeIntvl[ChnID][Status]>/proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                                                                                                                                                                                               |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[Status] on 打开 off 关闭                                                                                                                                                                                                                                                                                                                             |
| 举例   | echo statOutputTimeIntvl 0 on> /proc/mi_modules/mi_vdec/mi_vdec0<br>[MI VDEC PROCFS]: Output ChnID: 0, CurTime: 6413830139, TotalFrmCnt: 19, AvgTir<br>49999, MaxTimeIntvl: 50021, MinTimeIntvl: 49985<br>[Output ChnID] 通道号<br>[CurTime] 当前时间<br>[TotalFrmCnt] 输出帧次数<br>[AvgTimeIntvl] 每秒内所有帧输出平均时间间隔<br>[MaxTimeIntvl] 每秒内两帧输出最大时间间隔<br>[MinTimeIntvl] 每秒内两帧输出最小时间间隔。 |



|      |                                                                                                                                                                                                  |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 打开/关闭统计打印输入码流的时间戳                                                                                                                                                                                |
| 命令   | echo ChkInputFrmPts [ChnID] [Status] > /proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                         |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[Status] on 打开 off 关闭                                                                                                                                                      |
| 举例   | echo ChkInputFrmPts 0 on > /proc/mi_modules/mi_vdec/mi_vdec0<br>[MI VDEC PROCFS]: Input ChnID: 0, TotalFrmCnt: 400, PTS: 3964389<br>[Input ChnID] 通道号 ;<br>[TotalFrmCnt] 输入帧次数 ;<br>[PTS] 码流时间戳。 |



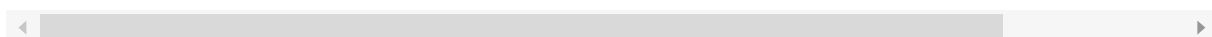
|      |                                                                                                                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 打开/关闭统计打印输出码流的时间戳                                                                                                                                                                                      |
| 命令   | echo ChkOutputFrmPts [ChnID] [Status] > /proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                              |
| 参数说明 | [ChnID] 通道号 [0~32]<br>[Status] on 打开 off 关闭                                                                                                                                                            |
| 举例   | echo ChkOutputFrmPts 0 on > /proc/mi_modules/mi_vdec/mi_vdec0<br>[MI VDEC PROCFS]: Output ChnID: 0, TotalFrmCnt: 154, PTS: 4039893000<br>[Output ChnID] 通道号 ;<br>[TotalFrmCnt] 输出帧次数 ;<br>[PTS] 码流时间戳。 |



|      |                                                                                                                                                                                                    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 将写入码流buffer的码流同时存文件                                                                                                                                                                                |
| 命令   | echo dumppbs [Path] [MoreChnID] > /proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                                |
| 参数说明 | [Path] 存储路径<br>[MoreChnID] 通道号 [0~32] 可以设置多路，如 0 1 2 3                                                                                                                                             |
| 举例   | echo dumppbs /mnt 0 > /proc/mi_modules/mi_vdec/mi_vdec0<br>[MI WRN ]: _MI_VDEC_IMPL_InjectBuffer[737]: Chn(0) Dump Es Buffer, Size:5765, (0x01 0x61)<br>在/mnt下产生 chn_0_h264_dump_vdec.es 文件，为es流文件 |



|      |                                                                                                                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 将解码出的图像同时存文件，存3帧                                                                                                                                                                                                                                                                                                                     |
| 命令   | echo dumpfb [Path] [MoreChnID] > /proc/mi_modules/mi_vdec/mi_vdec0                                                                                                                                                                                                                                                                   |
| 参数说明 | [Path] 存储路径<br>[MoreChnID] 通道号 [0~32] 可以设置多路，如 0 1 2 3                                                                                                                                                                                                                                                                               |
| 举例   | echo dumpfb 0 /mnt > /proc/mi_modules/mi_vdec/mi_vdec0<br>[MI WRN ]: _MI_VDEC_IMPL_DebugWriteFile[3494]: dump<br>file(/mnt/chn_0_h264_dump_vdec[720_576_736]_0.yuv) v1 ok .....[len:635904]<br>/mnt下产生 chn_0_h264_dump_vdec[720_576_736]_0.yuv，<br>chn_0_dump_vdec[720_576_736]_0_luma.yuv，chn_0_dump_vdec[720_576_736]_0<br>各3帧，共9帧 |



|      |                                                                                                                            |
|------|----------------------------------------------------------------------------------------------------------------------------|
| 功能   | 将当前码流buffer中的所有数据存储在文件中                                                                                                    |
| 命令   | echo dumpbsb [Path] [ChnID] > /proc/mi_modules/mi_vdec/mi_vdec0                                                            |
| 参数说明 | [Path] 存储路径<br>[ChnID] 通道号 [0~32]                                                                                          |
| 举例   | echo dumpbsb /mnt 0 > /proc/mi_modules/mi_vdec/mi_vdec0 Dump Chn(0)<br>ES Data Now<br>在/mnt下产生 chn_0_h264_dump_now_vdec.es |